

〔加餐〕Meta-Harness：Agent 优化 Agent + 课程总结

定位：前沿展望 + 双主线回顾。选讲，视时间决定展开深度。

从六讲的共同主题切入

六讲手工拆解了 Agent 的引擎盖：

讲次	拆解的部件
第一讲	System prompt、OODA 循环框架
第二讲	工具调用协议、Action space 设计
第三讲	上下文管理、记忆压缩策略
第四讲	信息粒度控制、ACI 设计
第五讲	架构选择（流水线 vs 循环）
第六讲	协同编排、多 Agent 通信
第七讲	权限引擎、Hook 系统、CLAUDE.md

这些设计目前都是人类工程师手工调优的——调 system prompt 的措辞、设计工具 schema 的参数、选择上下文压缩的时机和策略、配置循环的退出条件。

自然的问题：**这些 harness 设计本身能否被自动优化？**

什么是 Harness？

〔Meta-Harness（Lee et al., Stanford, 2026）：自动化 Harness 工程〕

论文给出了一个精确的定义（Section 3）：

Harness 是包裹在 LLM 外面的代码——决定什么信息**存储**、什么信息**检索**、什么信息**呈现**给模型。

用我们课程的语言翻译：

Harness 的组成部分：

- System prompt ← 第一讲拆解的
- 工具定义 (name + schema) ← 第二讲拆解的
- 上下文压缩策略 ← 第三讲拆解的
- 循环控制逻辑 ← 第三讲拆解的 (queryLoop 的 12 退出 + 7 继续)
- 信息粒度控制 ← 第四讲拆解的 (Read offset/limit, Grep head_limit)

换句话说：我们花了六讲拆的东西，统称 "harness"。

论文的核心发现 (Section 1)：

改变 harness 可以在同一个模型、同一个 benchmark 上产生 6 倍性能差距。

“Harness matters as much as the model itself.” 模型是引擎，harness 是变速箱——同一台引擎，配不同的变速箱，输出功率天差地别。

Meta-Harness 的搜索循环

既然 harness 这么重要，能不能让 Agent 自己来优化 harness？

Meta-Harness 的答案是：用一个 **coding agent** (Claude Code + Opus 4.6) 作为 proposer，通过搜索循环自动发现更好的 harness。

三步循环 (论文 Algorithm 1)：

初始化：

- 一组 seed harness (人类手工设计的基线方案)
- 一个空的文件系统 D

for t = 1 to N:

Step 1 · Proposer 读取文件系统 D

- D 包含所有历史候选的：
 - 源码 (harness 的 Python 代码)
 - 得分 (在评测任务上的表现)
 - 完整执行轨迹 (每次运行的 prompt、tool call、model output)
- Proposer 每次迭代读取中位数 82 个文件

Step 2 · Proposer 提出新 harness

- 单文件 Python 程序
- 基于对历史失败轨迹的分析，做针对性修改

关键设计：通过文件系统暴露完整历史

Meta-Harness 与其他 prompt 优化方法（OPRO、TextGrad、OpenEvolve 等）的核心区别是**信息量**：

方法	给 proposer 看什么	每次迭代 token 量
OPRO	(solution, score) 对	~200
TextGrad	当前候选的文本反馈	~15K
OpenEvolve	程序数据库 + 评估分数	~22K
Meta-Harness	所有历史候选的源码、分数、完整执行轨迹	~10M

信息量差了三个数量级。为什么需要这么多？

因为 **harness 优化是因果推理，不是模式匹配**。一个 harness 失败了，仅知道“得了 30 分”没有用——proposer 需要看到完整的执行轨迹才能推理出**为什么**失败：是 prompt 引导错了方向？是工具输出被截断丢了关键信息？是上下文压缩把重要细节吃掉了？

消融实验（论文 Table 3）证实了这一点：

	Scores	Code	Summary	Traces	Median	Best
Scores Only	✓	✓	✗	✗	34.6	41.3
Scores + Summary	✓	✓	✓	✗	34.9	38.7
Meta-Harness (full)	✓	✓	-	✓	50.0	56.7

只给分数 → 34.6；给分数+摘要 → 34.9（几乎没提升）；给完整轨迹 → 50.0

摘要反而不如原始轨迹——“Access to raw execution traces is the key ingredient for enabling harness search.” 这和第三讲的上下文压缩形成有趣的对比：对 Agent 执行任务来说，压缩是必要的（窗口有限）；但对 harness 优化来说，压缩是有害的（诊断细节不可丢）。

TerminalBench-2 结果：Agent 优化 Agent 的实战表现

TerminalBench-2 是一个 89 道高难度编程任务的基准测试——需要长期自主执行、处理复杂依赖、运用领域知识。这是一个被各方团队积极竞争的公开排行榜。

Meta-Harness 在两个模型上的表现（论文 Table 7）：

Claude Opus 4.6

Harness	Pass Rate
Claude Code（原生 harness）	58.0%
Terminus 2	62.9%
Mux	66.5%
TongAgents	71.9%
Terminus-KIRA（手工设计第一名）	74.7%
Capy	75.3%
ForgeCode	81.8%
Meta-Harness（自动搜索）	76.4%

Meta-Harness 超过了 Terminus-KIRA（手工设计排行第一），排名 #2。唯一更高的 ForgeCode（81.8%）依赖了公开代码之外的组件，结果不完全可复现。

Claude Haiku 4.5

Harness	Pass Rate
OpenHands	13.9%
Claude Code（原生 harness）	27.5%
Terminus 2	28.3%
Mini-SWE-Agent	29.8%
Terminus-KIRA	33.7%
Goose（手工设计第一名）	35.5%
Meta-Harness（自动搜索）	37.6%

在弱模型上，Meta-Harness 排名 #1——击败所有手工设计的 Haiku 4.5 方案。

两个模型的提升幅度对比：

Opus 4.6: CC 原生 58.0% → Meta-Harness 76.4% (+18.4%)
Haiku 4.5: CC 原生 27.5% → Meta-Harness 37.6% (+10.1%)

相对提升：
Opus 4.6: +31.7%
Haiku 4.5: +36.7%

模型越弱，harness 优化的相对收益越大。这和我们整个课程用小模型做 Demo 时观察到的现象一致——小模型对 system prompt 措辞、工具设计、上下文管理的敏感度远高于大模型。大模型有足够的智能“自救”（绕过不完美的 harness），小模型没有这个余量——harness 的质量直接决定了它的表现上限。

这也回答了一个实践问题：**如果你在用弱模型做 Agent（成本考虑），花时间优化 harness 的 ROI 比换更强的模型更高。**

CC 自身的 Harness 旋钮：同一模型的多档调节

Meta-Harness 论文说“改变 harness 可产生 6 倍性能差距”。CC 的源码提供了一个更贴近日常的实证：**即使不改 harness 代码，仅调整运行时参数，同一个模型的行为也会显著不同。**

CC 源码实证（cli.js）：CC 暴露了两个关键的 harness 旋钮：

旋钮一：Effort Level（思考强度）

级别	效果	适用场景
low	快速回答，浅层推理	简单查询
medium	默认平衡	日常编码
high	深度推理，更长 thinking	复杂 bug 修复
max	最深推理（仅 Opus 4.6）	架构决策、困难问题

同一个 Opus 4.6 模型，low effort 可能 1 轮完成简单任务，max effort 则会在 thinking 阶段花更多 token 做深度分析。**Effort level 本质上是 harness 控制模型在 Orient 环节投入多少推理资源——对应 Boyd 框架中“每轮 OODA 的深度 vs 整体循环的速度”的 trade-off。**

旋钮二：Fast Mode（速度 vs 成本）

模式	Input 价格	Output 价格	效果
Normal Opus 4.6	\$5/M	\$25/M	标准速度
Fast Opus 4.6	\$30/M	\$150/M	更快输出（同一模型，6x 价格）

Fast mode 不是换模型——是同一个 Opus 4.6，用 6 倍价格换更快的 token 生成速度。这直接对应 Boyd 的“循环转速”：**付更多钱让每轮 OODA 跑得更快，整体循环转速提升**。当 fast mode 遇到 429 rate limit 时，CC 自动 cooldown 降回普通模式——harness 在运行时动态调节速度/成本平衡。

这两个旋钮共同说明：**harness 不只是静态的代码，也包括运行时的参数调节**。Meta-Harness 论文搜索优化的是 harness 代码（system prompt、工具设计），但 effort level 和 fast mode 展示了另一个优化维度——运行时配置。完整的 harness 工程包括两者。

双主线回顾

主线一：任务结构（Localization → Repair → Validation）

所有编程 Agent——无论架构选择——都在完成同一个任务结构：

- 第一讲 定义 L-R-V 框架（Agent vs Chat 的质变）
- 第二讲 Act 环节基础设施（工具调用协议，四个流派）
- 第三讲 修 Bug 实战走完一遍 L→R→V（两个 bug，两轮循环）
- 第四讲 Localization 精细化（ACI 粒度控制，三种定位策略）
- 第五讲 Agentless 流水线覆盖 L+R+V 全链路（无循环的完整方案）
- 第六讲 层次化 L→R→V（Meta-L 拆解 → Worker 独立 L-R-V → Meta-V 集成）

主线二：OODA 循环的演进

每篇论文优化了决策循环的一个环节：

- ReAct → 建立闭环循环 （第一讲：从开环到闭环的范式革命）
- Toolformer → 增强 Act （第二讲：token 级 vs turn 级工具调用）
- Reflexion → 增强 Orient （第三讲：用自然语言替代梯度）
- SWE-agent → 优化 Observe （第四讲：ACI，信息粒度的精确控制）
- Agentless → 质疑循环本身 （第五讲：流水线 ≈ Agent 的 70-80%）
- Agent Team → 编队 OODA （第六讲：嵌套循环，星型拓扑 O(N)）
- Meta-Harness → 自动优化循环本身 （加餐：Agent 优化 Agent）

从“建立循环”开始，到“自动优化循环本身”结束——课程的终点恰好是下一个起点。

完整认知地图

	Localization	Repair	Validation
Observe (第四讲)	★ SWE-agent ACI ★ AutoCodeRover AST ★ Agentless 漏斗		
Orient (第三讲)	★ 测试报错后的 Thought ★ 上下文压缩	★ Reflexion 经验积累	★ 死循环诊断
Act (第二讲)		★ search/replace ★ CodeAct	
全局 (第五讲)	★ Agentless 三层漏斗 ← 流水线覆盖 L + R + V 全链路 →	★ 多候选采样	★ 测试筛选
编队 (第六讲)	★ Meta-L Lead 拆解 ← 星型拓扑，O(N) 协调开销 →	★ 并发 Worker 独立	★ 集成 Lead 汇总
评测	SWE-bench（定义端到端评测基准）		
★ 元优化 (加餐)	← Meta-Harness：自动搜索优化 harness 代码本身 → 覆盖 system prompt / 工具设计 / 上下文策略 / 循环控制		

四句总结

- 循环转速决定胜负** OODA：快速的“足够好”决策胜过迟到的“完美”决策。
ReAct 建立闭环，Reflexion 化失败为经验，Agent 的核心优势在于能从错误中学习。
- Harness 工程决定上限** 工具设计、上下文管理、循环控制——同一模型换 harness 可差 6 倍。模型是引擎，harness 是变速箱。我们花六讲拆解的每一个部件，都是 harness 的一部分。
- 协同架构决定规模** 星型拓扑 O(N) 协调：Worker 之间不通信，只向 Lead 汇报。
Boyd 推论三——协调开销是编队 OODA 的瓶颈，简单拓扑优于复杂拓扑。

安全机制决定底线 权限引擎、Hook 系统、CLAUDE.md——三层防线确保 Agent 的行为边界可控。结构化工具集是精细权限控制的前提，能力越大，边界越重要。